

Lab overview

This lab provides me with hands-on experience using the Amazon Web Services (AWS) Command Line Interface (CLI), Amazon Elastic Compute Cloud (Amazon EC2), and user data. I'll learn how to interact with the Amazon EC2 service using the AWS CLI to create an Amazon EC2 instance. Once the Amazon EC2 instance is created, I'll retrieve the IP address so I can verify the user data script ran successfully. I'll verify this by visiting the website hosted on the Amazon EC2 instance as it is a step in the user data script.

Objectives

By the end of this lab, I will be able to do the following:

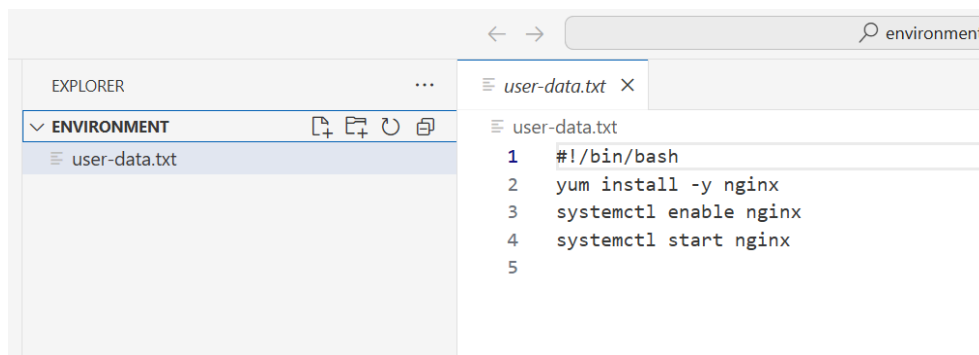
- Review the user-data.txt script to learn what it does.
- Update the Amazon EC2 CLI command to successfully launch an instance.
- Run the Amazon EC2 CLI command to retrieve the public IP address of the Nginx web server.
- Verify the NGINX web server is accessible and accepting web requests.

Task 1: Review the user data file

In this task, I'll connect to an AWS Code Editor environment that is used as the Integrated Development Environment (IDE) to review the user data file used in this lab. Amazon EC2 user data is one option to bootstrap an Amazon EC2 Linux instance with specific packages, to copy over files, set permissions, and other configurations I may want automated when creating an Amazon EC2 instance. After connecting to the Code Editor instance, I'll review the user data file to learn what it does during the bootstrapping process.

Task 1.1: Connect to the Code Editor environment

I'll connect to the AWS Code Editor environment provisioned as part of this lab. This is the IDE that I'll use to review the user data file and run an AWS CLI command to create an Amazon EC2 instance.



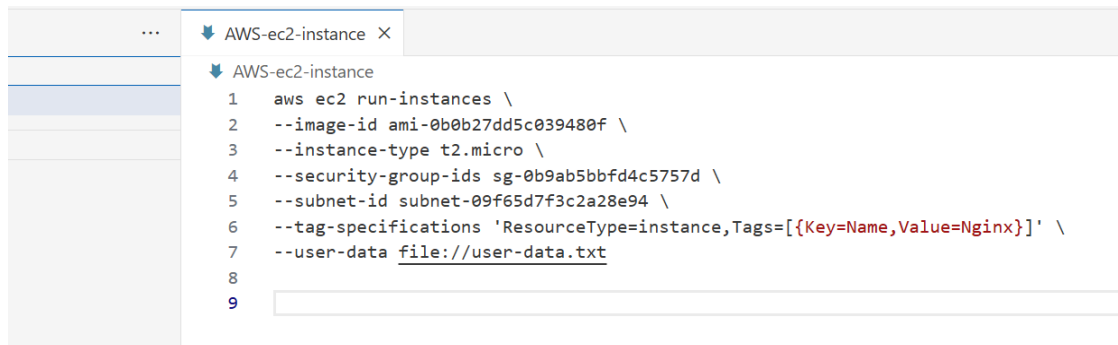
Below the specific commands are broken down to explain their purpose:

- **`#!/bin/bash`:** This is called a *shebang* and indicates that the script should be interpreted and ran using the bash shell.
- **`yum install -y nginx`:** This command uses the *yum* package manager (specific to certain Linux distributions including Amazon Linux) to install the Nginx web server. The `-y` flag is used to automatically answer yes to any prompts during the installation.
- **`systemctl enable nginx`:** This command allows the Nginx service to start automatically upon system boot. So, whenever the Amazon EC2 instance is restarted, Nginx is started as well.
- **`systemctl start nginx`:** This command starts the Nginx service immediately after it's installed, making the web server active and ready to serve web pages.

I have successfully connected to the AWS Code Editor environment and reviewed the user data file to understand what it is used for when creating an Amazon EC2 instance. It installs Nginx and configures this service to run when the Amazon EC2 instance is rebooted. It also starts the service so that the instance can immediately start returning web pages to web requests.

Task 2: Update the AWS CLI command to Create an Amazon EC2 instance

In this task, I'll update the Amazon EC2 AWS CLI command for the missing parameters to be able to successfully create an Amazon EC2 instance. The command I'll update is the `aws ec2 run-instance` command. The command is missing the value for the `--image-id`, the `--security-group-ids`, and the `--subnet-id`. I'll provide these missing details and then run the command to create the Amazon EC2 instance.



```
1  aws ec2 run-instances \
2  --image-id ami-0b0b27dd5c039480f \
3  --instance-type t2.micro \
4  --security-group-ids sg-0b9ab5bbfd4c5757d \
5  --subnet-id subnet-09f65d7f3c2a28e94 \
6  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Nginx}]' \
7  --user-data file:///user-data.txt
8
9
```

This command creates an Amazon EC2 instance with specific configurations. See a full breakdown of each parameter below.

Breakdown of each parameter in the command:

aws ec2-run-instances: This is the AWS CLI command to initiate the process of launching EC2 instances.

--image-id : This option specifies the Amazon Machine Image (AMI) ID of the instance I want to use.

--instance-type t2.micro: This specifies the instance type I want to launch. In this case, it's a *t2.micro* instance type, which is a low-cost, general-purpose instance type with modest resources. It is great for proof-of-concepts, but not what I would want to use for a production web server.

--security-group-ids : This option specifies the security group IDs for the instance. This is used to allow incoming web traffic to access my Nginx web server.

--subnet-id: This option specifies the subnet ID where the instance is launched.

--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Nginx}]': This option allows me to specify tags for the launched instance. In this case, a tag with a key of *Name* and a value of *Nginx* is added to the instance.

--user-data file://user-data.txt: This option allows me to provide user data to the instance. User data can be used to run scripts or perform certain configurations when the instance starts. In this case, the user data is being read from a file named *user-data.txt*.

Once I update the missing values for the **--image-id**, **--security-group-ids**, and the **--subnet-id** parameters, I'll copy the updated code snippet and paste it into the AWS Code Editor terminal and press the **Enter** key to run the command, which gave the following output:

```
[ec2-user@ip-10-10-10-136 environment]$ aws ec2 run-instances \
--image-id ami-0b0827d05c839480f \
--instance-type t2.micro \
--security-group-ids sg-0b9ab5bbfd4c5757d \
--subnet-id subnet-09f65d7f3c2a28e94 \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Nginx}]' \
--user-data file://user-data.txt
{
  "ReservationId": "r-026c337f07059ef17",
  "OwnerId": "460836325468",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      :
    }
  ],
  "ReservationId": "r-026c337f07059ef17",
  "OwnerId": "460836325468",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
```

```
{
  "Architecture": "x86_64",
  "BlockDeviceMappings": [],
  "ClientToken": "4790273a-f742-4f15-9f91-cfe3e9491245",
  "EbsOptimized": false,
  "EnaSupport": true,
  "Hypervisor": "xen",
  "NetworkInterfaces": [
    {
      "Attachment": {
        "AttachTime": "2026-07-04T07:38:19+00:00",
        "AttachmentId": "eni-attach-09bda30f15075350",
        "DeleteOnTermination": true,
        "DeviceIndex": 0,
        "Status": "attaching",
        "NetworkCardIndex": 0
      },
      "Description": "",
      "Groups": [
        {
          "GroupId": "sg-0b9ab5bbfd4c5757d",
          "GroupName": "Web Instance For Student SG"
```

This output, created in the JSON format, provides essential information about the launched Amazon EC2 instance, including its identifiers, networking details, and status. However, it does not provide output for the public IP address.

I successfully updated the missing values required to be able to create an Amazon EC2 instance and then created the Amazon EC2 instance using the appropriate Amazon EC2 AWS CLI *run-instances* command.

Task 3: Verify the user data ran successfully

In this task, I'll ensure the user data worked as expected by opening the Nginx web server running locally on the Amazon EC2 instance I just created. To do this, I'll need to obtain the *public IP address* of the instance.

Task 3.1: Run the Amazon EC2 AWS CLI command to retrieve the public IP address of the Nginx web server

There is an Amazon EC2 AWS CLI command that generates output with the public IP address for my Amazon EC2 instance. If the user data was defined properly, it has installed the Nginx web server and started it. So, when I open a browser to this instance's publicly accessible IP address it should return the default Nginx web page.

To retrieve only the **public IP address** for the instance with a tag where the **name** value is set to **Nginx**, I'll run the following command:

```
aws ec2 describe-instances --filters "Name=tag:Name,Values=Nginx" --query 'Reservations[].Instances[].PublicIpAddress'
```

```
[  
  "54.191.202.247"  
]
```

With this query, it returned only the *public IP address* value I was after and no other details.

Task 3.2: Verify the Nginx web server is functional

Now that I have the *public IP address* for the Amazon EC2 instance, I can confirm the Nginx web server is working by accessing the Nginx web server's default home page. In a new browser tab I'll paste the **public IP address** I copied earlier into the address bar to be taken to the Nginx default home page.

I should see a message reading as show below:

Welcome to nginx!

After the instance was created, I obtained the public IP address of the instance using the `aws ec2 describe-instances` AWS CLI command with a couple options. I also verified the user data was processed successfully by viewing the default home page for the local Nginx web server running on the instance.

Conclusion

I have successfully done the following:

- Reviewed the user-data.txt script to learn what it does.
- Updated the Amazon EC2 CLI command to successfully launch an instance.
- Ran the Amazon EC2 CLI command to retrieve the public IP address of the Nginx web server.
- Verified the NGINX web server is accessible and accepting web requests.

